

Unified Software Requirements Specification (SRS) & Project Plan

Part I

2 Personnel & Roles

Name	McMaster Email	Primary Technical Disciplines
Bobby Abtahi	abtahib@mcmaster.ca	Machine Learning, Software
Guneev Arora	arorag12@mcmaster.ca	Machine Learning
Evan Moon	moone1@mcmaster.ca	UI/UX, Software
Kamalpreet Singh	singk17@mcmaster.ca	Embedded Programming, UI
Natasha Taylor	taylon17@mcmaster.ca	Software Engineering, Machine Learning

3 Definitions, Acronyms, & Abbreviations

Technical Term	Explanation
UI	Stands for “User Interface”
OS	Stands for “Operating System”
Win	Abbreviation of “Windows” the OS by Microsoft
PR	Stands for “Pull Request”
GPU	Stands for “Graphics Processing Unit”

4 Contribution Table

Name	Parts Worked On	Estimated Contribution Percentage
Bobby Abtahi	all	20%
Guneev Arora	all	20%
Evan Moon	all	20%
Kamalpreet Singh	all	20%
Natasha Taylor	all	20%

5 Executive Summary & Purpose

We aim to develop a camera-based mouse that uses computer vision to track the movement and positions of a user’s hands. The goal is to create a functional alternative to current mouse technology by building a plug-and-play style input peripheral that steps away from more traditional human-computer-interface designs and feels more natural to use. Interaction via a traditional mouse is familiar and effective and has become second-nature to most people, but it does have trade-offs. Most mice use some form of optical tracking

and require an adequate surface for them to do so, the accuracy of their tracking can be limited depending on a user's environment, and depending on a user's position and setup they can be inconvenient to use or worse cause joint pain in the wrist and arm. Our proposed solution is a mouse that ditches this paradigm and instead opts to track the motions of a user's hand and translate that into input commands allowing the user to interact with their computer without having to touch an input device at all. Furthermore, for the system to feel smooth and useable it must have high accuracy, low latency, and be easy to set up. The intended impact of the project is to demonstrate that a camera-based hand-tracking input system is a viable alternative to regular pointing devices. The project should enable users to interact effectively with their computers in scenarios where a regular mouse is inconvenient to use or is not usable due to accessibility concerns.

Part II

6 Stakeholders & User Personas

Stakeholders

- Primary Clients
 - As touched on in the executive summary, the introduction of this device can have a direct effect on multiple groups, who we expect to use this product regularly.
 - Computer Users with Disability/Ergonomic Concerns
 - * This device introduces the benefit of being accessible to a wider scope than a traditional mouse. Using a mouse typically requires a user to be sat in an upright position with a table/flat surface at a comfortable height above their lap. This set up raises many concerns for a different subsets of the population.
 - * Left-handed users: This device is not centered around right-hand dominant users, as opposed to the traditional mouse.
 - * Sufferers of Carpal Tunnel Syndrome/Arthritis/Repetitive Stress Injuries: An input that requires hand waving and gesturing rather than the typical outstretched "hand-claw" may provide relief.
 - * Sufferers of tremors: This device requires less fidelity than a typical move and click mouse.
 - * Wheelchair users: May have difficulty using a mouse, especially if the desk cannot be pulled towards their torso.
 - * People with a right hand of limited to no use: The device can be operated by use of either hand separately.
 - Tech Enthusiasts
 - * The computer mouse has been widely adopted since the 1980s, and its form factor has remained mostly unchanged. A novel idea such as Mouse++ will definitely have interest from the early-adopter tech enthusiast community.
- Secondary Stakeholders
 - Although these stakeholders may not use Mouse++ regularly, they will be indirectly affected by those who do use it, or the general introduction of this device into the market.
 - Family and Caregivers of Users: A device like this can provide autonomy to users who may have struggled to operate a computer using a mouse without the help of a caregiver.
 - Companies and Institutions: A viable alternative to a mouse that provides accessibility considerations would be on the radar of companies and institutions that purchase technology for their employees, students, etc.

- Legacy Computer Hardware Companies: Introducing a product in a space that has been dominated by computer mice for decades will be noticed by legacy companies like Logitech, Acer, Lenovo. Should our product ever be successful, these companies would need to pivot accordingly.

User Personas (End-User Archetypes)

1. A User with Mouse-related Pain

- Meet John
 - Age 22
 - Engineering Student at McMaster University
- John's Routine
 - John spends most of the day at home, seated at his desk using his computer
 - He spends 5-7 hours on the computer doing schoolwork and personal projects.
 - This involves using engineering software, reading lecture slides, and reviewing documents.
 - Once his work is done, he'll have 2-3 hours of leisure time, where he browses the internet and watches TV shows.
- John's Frustration
 - Constant use of his mouse has led to persistent pain in his right palm and wrist.
 - He has been advised to reduce repetitive stress on his right hand by a couple hours a day.
- John's Goal
 - He wants to be able to continue his regular routine, as limiting his computer time is not feasible.
- How Mouse++ Could Help
 - John can use Mouse++ to scroll through, click, and navigate his lecture slides, documents, and websites.
 - He may still use his keyboard, but a lot of the repetitive workload on his right hand has been offloaded to more ergonomic Mouse++ gestures.

2. A Tech Enthusiast

- Meet Jane
 - Age 30
 - Works as an IT Specialist at Rogers
- Jane's Routine
 - Jane works full time at Rogers, maintaining their data centre operations.
 - In her free time, Jane loves finding cutting edge tech online and buying it for personal use.
 - She has her own blog and posts a review about every new product she tries.
- Jane's Frustration
 - Jane purchases innovative tech in different product categories: folding phones, curved high-refresh rate displays, smart glasses, etc.
 - But she finds that input devices, especially the mouse, have changed very little in comparison.
 - Innovations like vertical designs, or adding programmable buttons are not exciting.
- Jane's Need
 - She is looking for exciting new ways to interact with her computer, beyond the traditional computer mouse.
- How Mouse++ can help
 - Mouse++ serves as a new exciting iteration of the mouse.
 - Jane would be eager to test and review this new way of interacting with her computer.

7 Constraints & Assumptions

Hardware Requirements

- Camera Module
- Microcontroller
 - The computer vision processing will be done on a microcontroller.
 - This gives a standard runtime environment so that we can fine-tune the system and performance to give an equivalent user-experience regardless of the computer the user is attaching the device to.
 - This also places limitations on the size of ML/CV models that we can run as they must be able to run efficiently and accurately on a small microcontroller.

Target Operating Systems: The device and control software should be compatible with the following operating systems:

- Windows 10/11 (x86 64-bit systems)
- Linux (x86 64-bit systems)
 - Linux covers a wide-range of platforms, but we would target full support for systems that use Wayland and limited support outside of Wayland.

This project does not have costs outside of the camera + microcontroller hardware costs. User data will be processed locally on the device and will not be collected or transmitted off-device.

8 Functional Requirements

P0

Req 1. Hand and Landmark Prediction

- Input: Camera input
- Output: List of 21 landmark (x,y) positions on hand if hand detected (e.g. fingers, knuckles, palm)
- Data flow: Camera → YOLO model → Landmark detection model

Req 2. Computer Vision System detects and classifies hand poses and motions - *Backend*

- Input: Hand landmarks from YOLO model
- Output: Classified hand poses and motion sent to OS
- Data flow: Camera → Computer Vision System → Device Link → Driver

Req 3. Driver moves cursor, with movement corresponding to hand motion detected by CV System

- Input: Chosen hand landmark (x,y) positions
- Output: Movement of cursor on computer UI
- Data flow: Computer Vision System → Device Link → Driver → OS

Req 4. Driver executes mouse clicks corresponding to hand poses detected by CV System

- Input: output from cv system
- Output: Right/left click interactions occur on user's computer
- Data flow: Computer Vision System → Device Link → Driver → OS

Req 5. Must include drivers for Win 10/11

P1

Req 1. Control Software - *Frontend*

- Application allows user to control Mouse++ settings/preferences, edit or add to the Gesture/Pose Library

Req 2. Gesture/Pose Library

- Input: Hand Pose/Gesture
- Output: Gesture/Pose Library stores a mapping of known/assigned gestures and poses to cursor motions and actions, and actions on the user's computer are triggered by their corresponding gesture/pose in the Library
- Data flow: Control Software $\leftrightarrow$ Driver $\leftrightarrow$ Computer Vision System.

Req 3. Screen capture target selection

- Input: Screen Area selection
- Output: The tracked inputs are contained/scaled within the selected screen bounds
- Data flow: Control Software $\leftrightarrow$ Driver

Req 4. Camera Operating bounds

- Input: Camera Operating bounds selection
- Output: The camera should exclude data collected from outside bounds
- Data flow: Control Software $\leftrightarrow$ Driver $\leftrightarrow$ Computer Vision System

Req 5. Automation/Scripting system

- Input: Visually defined automation/macro scripts
- Output: Control software should be able to associate scripts with a hand gesture and should be able to execute them
- Data flow: Script Definition $\rightarrow$ Control Software $\leftrightarrow$ Driver

Req 6. Control Software should have builds for Win 10/11 and Linux

Req 7. Must include drivers for Linux

P2

Req 1. Include drivers for Mac OS (specifically targeting Apple Silicon chips)

Req 2. Control Software should have builds for Mac OS (specifically targeting apple silicon chips)

Req 3. Firmware updates via the control software

- The control software should be able to push software updates for the Computer Vision system to the microcontroller

P3

- Virtual Keyboard software to allow the user to input text in an efficient way
- Voice Commands, through a microphone on the microcontroller that can trigger an action in the Control Software
- Handwriting text input tool, similar to the Windows Ink feature, allowing the user to draw text and use OCR over it as input

9 Data Strategy & Quantitative Metrics

Datasets

- HaGRID - HAnd Gesture Recognition Image Dataset
 - https://github.com/hukenovs/hagrid/tree/Hagrid_v2-1M
 - 33 pre-labelled hand-gesture classes
 - 1,086,158 total images
 - Publically licensed with attribution and conditions reserved, specified in a license file in the repository
- YOLO11n-pose Hand Keypoint Detection
 - <https://github.com/chrismuntean/YOLO11n-pose-hands>
 - Existing training pipeline for hand-skeleton recognition similar to the outputs of Google's MediaPipe-hands model
 - Supports PyTorch
 - APGL 3.0 (inherited from YOLO11n)

Metrics

- Hand tracking should achieve 15 detections per second
- Gesture recognition success rate (test in different lighting and distances from the camera)
- Microcontroller CPU/RAM/TEMP usage
- Host computer CPU/RAM usage
- Latency between hand movement and UI response

10 Non-Functional Requirements

- The Control Software is the only component that contains a UI, it should follow WCAG 2.2 level AA guidelines, we likely will not be building it as a web app so WCAG will not apply directly but it is a good benchmark to follow for all UIs regardless
- The Mouse input events should achieve a poll rate of 15 updates per second.
- Control software should only change settings on a Mouse++ device that is currently attached to the computer it is running on.
- The Mouse++ device should not receive inputs / setting changes from, nor transmit input events to, anything other than the device drivers Mouse++ has provided to the currently connected computer.
- Privacy
 - No usage-statistics or user data should be collected or transmitted.
 - All camera data should be kept on the microcontroller, and it should be a closed system such that bad actors cannot remotely access the camera.
 - The camera should not save any recordings.
 - When the camera's video feed needs to be made available to the Control Software (for bounding box selection or gesture/pose setup), it should be visible obvious to the user that this is being done and the stream should close once the control software exits (intentionally or not). This feed should not be accessible to anything other than the control software

- The system should be fault tolerant such that unexpected failures should keep the Mouse++ device and the attached computer in a recoverable state.
 - The Mouse++ device should have some form of restart mechanism (both restarting the Computer Vision System or the entire microcontroller) if it runs into an error, it should attempt this restart automatically
 - Errors must be handled such that a driver error will not cause the computer to crash.
 - Unexpected errors in the Control Software must not place the Microcontroller in an unrecoverable state, and restarting the Control Software should allow the user to complete or restart the action they were taking.
- The three components of the project (Computer Vision System, Device Drivers, and Control Software) should be maintainable and modular such that changes (improvements, features, patches) can be component-specific and not require full software upgrades to all 3 when unnecessary.

11 Risk Management & Feasibility Analysis

Risk	Severity	Potential Impact	Mitigation Steps
Memory Leaks	High	Delays, overheating, hardware failure	Verify through Valgrind, review code by multiple people, and test.
Poor Environment Conditions	Low	Inaccurate inputs	Add warnings for these situations. Train the model using augmented data that includes them.
Device Link Failure	Medium	Poor connections, faulty wires, or Bluetooth interference can cause delays and a bad user experience.	Build redundant connections and warn users when failures occur.

Part III

12 Team Communication & Meeting Cadence

The group will meet to discuss updates and progress once a week with the entire group present. Meeting notes for these meetings will be taken by a group member assigned through a rotating schedule, and saved under the `/meetings` directory in the git repository. Group members are required to provide twice weekly updates to a shared communication channel in the form of a bulletin that the rest of the team can look at during the time in-between meetings if needed.

We do not have a supervisor so we will not need to schedule the 2/semester meetings with one.

We will schedule at least 1 meeting per semester with our TA to check-in and provide progress updates.

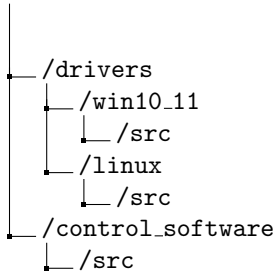
13 Engineering Workflow & Version Control

Repository structure

```

/
├── /docs
├── /meetings
├── /computer-vision
└── /src

```



The goal is to improve maintainability of the three components by having them be largely independent. As such they are split into their own directories in the repository and will also be given their own branches. Specifically the branch for one component will not include the source directories for either of the others. While unconventional this means that someone working on a specific component can only make changes to that component and also allows for a better view of the commit history on a per-component basis.

Workflow

The **main** branch will be locked so that no changes can be directly committed to it, nor can it be pushed to. Instead we will require PRs to be made for merges to the main branch and these PRs must receive approval from at least one group member other than the original author. *Note: Commits must not be squashed during a merge.*

The subbranches for each component will not be locked in this manner to prevent impeding development, but when merging changes made to these branches to main it is important for relevant commits to be cherrypicked so that only changes specific to a feature are included in a given PR. This keeps PRs smaller and easier to read and keep track of. While commits can be made directly to the subbranches it is recommended that work is done in separate branches that can then be merged to these subbranches. The separate branches should follow a predictable naming scheme, preferably `/<component>/<type>/descriptive-name`, so the branch for feature A on component B should be called `/B/feature/feature-A`.

The workflow for working on a component specific change then becomes:

1. Create branch specific to change, branching from the component's subbranch.
2. Merge branch into component's subbranch.
3. Cherrypick relevant commits to create PR to merge with main.
4. Get approval/review from at least 1 other member.
5. When approved, merge PR.

If instead a change is going to affect multiple components and it is important/relevant to the change to keep everything in one PR rather than 3 separate ones (this scenario will likely occur for late changes to the communication protocols/definitions) then a specific branch for the change can be branched from **main** itself. A PR must be created, approved, and merged, and then finally the changes must be cherrypicked back into their respective component subbranches. The workflow for a multi-component change becomes:

1. Create branch specific to the change, branching from **main**.
2. Create PR.
3. Get approval/review from at least 1 other member.
4. Merge PR.
5. Cherry pick changes relevant to each component from **main** back to each component's subbranch.

Pull Requests

PRs should be well-documented and easily legible. At the very least a PR must provide a description of the goal and relevant implementation details. Lengthy PRs should be structured into an easy to read format with headings for:

- Expected output/behaviour
- Current output/behaviour (optional)
- Implementation details
- Tested/considered error and edge cases

When possible a PR should include photos or screenshots to help understand the change. All PRs should be labelled depending on what components they affect. Relevant info should also be linked in a PR. This includes issues that the PR references or solves, and external documentation or articles if used to implement the PR (as well as internal documentation if relevant). The title of a PR should follow the following format: **[Component] [Type] Title** so a PR for a new feature A on Component B should be titled **[Component B] [Feature] Initial Implementation of Feature A**.

CI/CD Pipeline

We plan on using GitHub action workflows to automate pre-PR checks and to produce nightly builds. Some of the components (drivers and Control Software) will need to be compiled and so we would like tests built in to make sure that the changes in a given PR compile without errors and do not introduce compile errors to the codebase. For these components we would like to create nightly builds so that we can test latest changes frequently.

Issue Tracking

Issues should be opened when bugs, incorrect behaviour, or unexpected behaviour/cases are found. Issues should be labelled based on the component they affect. Issues should be titled in the following format: **[Component] [Type] Title** so an issue regarding an edge-case for feature A on Component B should be titled, e.g.: **[Component B] [Edge Case] Edge case in Feature A causes** Issues should then be referenced and closed when a PR fixes or addresses them. Closing an issue should only be done by the original author to ensure that whatever behaviour they noticed is properly handled by the fix before closing. Relevant information should be linked with each issue (documentation, code snippets, stack traces, etc.) as well as a detailed description. Any thoughts on the cause of an issue and/or possible solutions should be added in the comment thread of an issue rather than the issue description itself.

Agile

Our team plans to do an Agile-focused development plan. Our project is split into software, machine learning, and driver parts. Before each sprint, we plan the next feature set we would like to implement and then go through with a weekly sprint, after which we will conduct a review at our next meeting. All progress should be documented in PRs on GitHub, which should be ready for review or already reviewed.

14 Infrastructure & Compute Resources

The final deployment for this project will be local, so cloud resources are not required. Instead we will need compute resources when training/developing our computer vision models. Between the group we have a system with an Nvidia RTX 5070 and a system with an AMD Radeon 6600. These systems should suffice for testing/exploratory work, but we will likely need more compute to train the final models. In this scenario we will likely use the available McMaster GPU clusters or rent a compute cluster from a service like <https://rentgpu.org/gpus> which provides access to a wide range of GPUs at prices ranging from \$0.018 per hour to \$12.95 per hour. We will likely not need a top of the line GPU and so the costs associated with this compute will be largely minimal; we anticipate it being under \$50. Our anticipated hardware for training is an RTX 4090/5090,

15 Technology Stack & Architectural Rationale

- Computer Vision System

- The ML model will be designed in Python using the PyTorch package, as it has a mature ecosystem and integrates well with other packages. Frequently called functions will be re-written and jit-compiled using JAX to meet the non-functional latency requirement.
- We will train the model both locally and over rented cloud GPU instances, with 3 model stages to maximize performance. A small YOLO model will be in charge of binary hand detection, a custom trained efficient hand landmark/pose estimator will be used to model hand shape, and a small Graph Convolutional Network or Multi-Layer Perceptron model will convert the raw pose outputs into an embedding vector to be used for supporting custom gestures.
- The dataloader will use the huggingface Datasets package.
- Device Drivers
 - Will be written in C++, which gives a high level of control and is built to interface with existing device driver development kits.
 - The Win 10/11 driver will be written using the Windows Driver Kit (WDK) which is a software development kit designed to provide the necessary Windows library details to implement a device driver, and provides the tools required to sign, package, and build said driver.
- Control Software
 - C++ with Qt as a UI Framework.
 - Qt is highly portable and supported across most platforms, (supported on Windows/Linux/Mac OS).
 - Writing both the control software and the device driver in the same language makes it easier to implement communication between the two as it can be written as a library that is shared between both components.
 - Qt provides a wide range of UI support for video streaming, UI interactions and a well-featured event-loop system to handle user interactions as well as responses/messages from the device/device driver.

16 Proof of Concept Demonstration Plan

The PoC for November 23rd must have a rudimentary computer vision model that shows that the technology itself works (even if it is not finetuned yet). It must also have an example driver for at least one OS (either Win10/11 or Linux). We would also like to have a working communication system between the computer vision system, the device drivers, and the UI. We would like to set up a simple UI that can show that this communication is happening as expected. The UI can be extremely rudimentary and is meant as a barebones basis for the Control Software to be built on top of. The PoC must have a distributable binary for this UI so that we can assure that distributable builds of the Control Software are technically feasible.

17 Project Schedule & Gantt Chart

